PRACTICAL NO. 4

LLM-Based Chatbot

Development of a Domain-Specific College Assistant Chatbot
using Python, LangChain and Groq API

Student Name	_____
Class / Division	_____
Roll Number	_____
Subject	Generative Artificial Intelligence
Date	_____

Practical record / examination write-up

1. Aim

To develop a domain-specific conversational chatbot using a Large Language Model (LLM) and implement conversation history management using LangChain.

2. Objectives

- To connect a Python application with a Generative AI model through Groq API.
- To develop a chatbot for the college-information domain.
- To store college-related information in an external text file.
- To provide the college information to the LLM through a system prompt.
- To maintain conversation history and answer follow-up questions using previous messages.

3. Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM recommended.
- Internet connection for API access and model inference.

4. Software Requirements

- Operating System: Windows, Linux or macOS.
- Python 3.10 or later.
- Visual Studio Code or any Python IDE.
- Python libraries: langchain, langchain-groq and python-dotenv.
- Groq API key and an available Groq model.

5. Theory

A Large Language Model (LLM) is an artificial intelligence model trained on large amounts of text data. It can understand prompts and generate human-like responses.

A domain-specific chatbot is designed to answer questions within a selected subject area. In this experiment, the selected domain is college information. The chatbot uses a system prompt containing information read from an external text file. This file can contain college details, rules, policies, courses, admission information and examination guidelines.

LangChain message objects are used to manage the conversation. SystemMessage defines the chatbot's role and instructions, HumanMessage represents the user's input, and AIMessage stores the chatbot's response. Sending the message history to the LLM allows it to understand follow-up questions.

Note: This implementation places the complete text-file content in the system prompt. It is a simple context-based approach, not a Retrieval-Augmented Generation (RAG) system.

6. Main Components

Component	Purpose
Python	Implements the chatbot application.
Groq API	Provides access to a hosted language model.
ChatGroq	Connects LangChain to the Groq chat model.

Component	Purpose
SystemMessage	Defines the chatbot's role, domain and rules.
HumanMessage	Stores the user's message.
AIMessage	Stores the assistant's response.
college_info.txt	Stores college-related knowledge.
Conversation history	Maintains recent messages for context.

7. Project Folder Structure

```
College_Chatbot/
|
|-- main.py
|-- college_info.txt
|-- .env
```

The **main.py** file contains the chatbot program. The **college_info.txt** file contains college knowledge, and the **.env** file stores the Groq API key.

8. College Knowledge File (college_info.txt)

Create a file named **college_info.txt** in the same folder as main.py. Add verified information about the selected college. The following is sample content for demonstration; replace it with the college's actual and approved information.

```
COLLEGE INFORMATION

College Name: ALARD University

Location:
Hinjawadi, Pune, Maharashtra, India.

ABOUT THE COLLEGE:
ALARD University is an educational institution
located in Pune, Maharashtra.

ACADEMIC INFORMATION:
Refer to the official university website for
the current courses and academic programmes.

COLLEGE RULES:
1. Students must carry their college ID cards.
2. Students must maintain classroom discipline.
3. Students must follow college instructions.
4. Students must maintain cleanliness.
5. Students must follow examination guidelines.

ATTENDANCE POLICY:
Students must follow the official attendance
requirements specified by the university.

EXAMINATION POLICY:
1. Students must follow examination instructions.
2. Students must carry their examination hall ticket.
3. Students must maintain discipline during exams.

ADMISSION INFORMATION:
Admissions are subject to eligibility criteria
and university admission guidelines.

HOSTEL AND FEES:
Refer to the official university administration
for current hostel and fee information.

CONTACT:
Contact the university administration or consult
the official university website.
```

Important: The sample rules and details above are illustrative, not verified official ALARD University policies. Do not present them as official rules. Use verified details in the final practical demonstration.

9. API Key Configuration

Create a file named **.env** and add the API key obtained from the Groq Console:

```
GROQ_API_KEY=your_groq_api_key_here
```

Keep the API key private. Do not upload the .env file to a public repository. Groq API keys can be created at https://console.groq.com/keys.

10. Installation of Required Libraries

Open the terminal in the project folder and execute:

```
pip install langchain langchain-groq python-dotenv
```

11. Python Program (main.py)

```
import os
from dotenv import load_dotenv

from langchain_groq import ChatGroq
from langchain_core.messages import (
    SystemMessage,
    HumanMessage,
    AIMessage
)

# 1. Load API key
load_dotenv()
api_key = os.getenv("GROQ_API_KEY")

if not api_key:
    raise ValueError("Groq API key not found!")

# 2. Initialize the language model
llm = ChatGroq(
    model="openai/gpt-oss-20b",
    temperature=0.3,
    api_key=api_key
)

# 3. Read college information from text file
with open("college_info.txt", "r", encoding="utf-8") as file:
    college_data = file.read()

# 4. Define the system prompt
system_prompt = f"""
You are an AI College Assistant for ALARD University.

Use the following college information to answer
the user's college-related questions.

COLLEGE KNOWLEDGE:
{college_data}

Instructions:
1. Answer college-related questions only.
2. Use the provided college information.
3. Do not invent facts or college policies.
4. If information is unavailable, say:
   "Sorry, I don't have this information."
5. Answer politely and clearly.
"""

# 5. Initialize conversation history
chat_history = [
    SystemMessage(content=system_prompt)
]

print("\\n===== ALARD COLLEGE ASSISTANT =====")
print("Ask college-related questions.")
print("Type 'exit' to stop.\\n")

# 6. Chat loop
while True:
    user_input = input("You: ").strip()

    if user_input.lower() == "exit":
        print("Bot: Thank you! Goodbye.")
        break

    if not user_input:
        continue

    # Add user message to history
    chat_history.append(
        HumanMessage(content=user_input)
```

```
    )

    # Generate response using conversation history
    response = llm.invoke(chat_history)

    # Add AI response to history
    chat_history.append(
        AIMessage(content=response.content)
    )

    print("\\nBot:", response.content, "\\n")

    # Keep the system prompt and latest 10 messages
    if len(chat_history) > 11:
        chat_history = (
            [chat_history[0]] + chat_history[-10:]
        )
```

12. Algorithm

- Start the program.
- Load the Groq API key from the .env file.
- Initialize the LLM using ChatGroq.
- Read college information from college_info.txt.
- Create a system prompt containing the chatbot instructions and college information.
- Initialize conversation history with the SystemMessage.
- Accept a question from the user.
- Append the question as a HumanMessage.
- Send the conversation history to the LLM using invoke().
- Append the LLM response as an AIMessage and display it.
- Keep the system prompt and recent messages in history.
- Repeat until the user enters exit, then stop.

13. Execution

Run the program from the terminal:

```
python main.py
```

14. Sample Output

```
===== ALARD COLLEGE ASSISTANT =====
Ask college-related questions.
Type 'exit' to stop.

You: Where is our college located?

Bot: ALARD University is located in Hinjawadi,
Pune, Maharashtra, India.

You: What are the college rules?

Bot: According to the provided information,
students must carry their college ID cards,
maintain classroom discipline, follow college
instructions and examination guidelines.

You: What about attendance?

Bot: Students must follow the official attendance
requirements specified by the university.

You: What is the hostel fee?

Bot: Sorry, I don't have this information.

You: exit
Bot: Thank you! Goodbye.
```

The above is illustrative output. Actual wording may vary depending on the model response and the information in the text file.

15. Explanation of Important Code

load_dotenv() — Loads environment variables from the .env file.

os.getenv('GROQ_API_KEY') — Reads the API key without hardcoding it in the Python program.

ChatGroq() — Initializes the chat model through Groq.

open('college_info.txt', 'r') — Opens the college knowledge file in read mode.

file.read() — Reads the complete text-file content into a Python string.

SystemMessage() — Defines the chatbot's role and provides its instructions and context.

HumanMessage() — Represents a message entered by the user.

llm.invoke(chat_history) — Sends the conversation messages to the LLM and obtains a response.

AIMessage() — Stores the chatbot's generated response in conversation history.

chat_history.append() — Adds a new message to the conversation history.

16. Result

The domain-specific College Assistant Chatbot was developed successfully using Python, LangChain and Groq API. College-related information was loaded from an external text file and supplied to the LLM through a system prompt. Conversation history was maintained using HumanMessage and AIMessage, enabling the chatbot to respond to follow-up questions using previous conversation context.

17. Conclusion

This practical demonstrates how an LLM can be integrated into a Python application to build a domain-specific conversational chatbot. External text-file knowledge helps the chatbot answer college-related questions, while conversation history enables context-aware interaction.

18. Viva Questions and Answers

1. What is an LLM?
LLM stands for Large Language Model. It understands and generates human-like text.

2. What is a domain-specific chatbot?
A chatbot designed to answer questions about a particular subject or domain.

3. What is LangChain?
A framework used to build applications powered by language models.

4. What is Groq?
An AI inference platform that provides access to language models.

5. What is a system prompt?
Instructions that define the chatbot's role and behaviour.

6. What is HumanMessage?
It represents the user's input.

7. What is AIMessage?
It represents the AI-generated response.

8. What is conversation history?
It stores previous user and AI messages for conversational context.

9. What does invoke() do?
It sends messages to the LLM and returns its response.

10. Why use a text file?
To store and provide external domain-specific information.

11. Is this application RAG?
No. It supplies the complete text file as context rather than retrieving selected document chunks.

12. What is temperature?
A parameter that controls the randomness of generated responses.

19. Precautions and Limitations

- Keep the Groq API key private and store it in the .env file.
- Use verified college information; do not treat sample policies as official.
- The computer needs internet access to call the Groq API.
- The selected model must be available to the Groq account at the time of execution.
- The entire text file is sent as context, so keep it reasonably small to fit within the model's context limit.
- The text file is read when the program starts. Restart the program after editing the file.
- Conversation history is stored in memory only and is lost when the program exits.
- The chatbot may still make mistakes; verify important college policies with official sources.